1. Introduction. This example applies a function to every element of a slice *concurrently*, using a fixed pool of worker goroutines, and returns the results in the original order. It exercises the Go features a literate document most wants to show off nicely: generics, goroutines, channels, *sync.WaitGroup*, and a closure.

2. The program imports only the standard library and demonstrates *pmap* from *main*.

```
package main
import (
    "fmt"
    "sync"
)
⟨ The parallel-map function 3 ⟩
func main() {
    nums := []int{1, 2, 3, 4, 5, 6, 7, 8}
    squares := pmap(nums, 3, func(n int) int { return n * n })
    fmt.Println(squares)
}
```

3. The parallel map. The signature is fully generic: *pmap* maps a $[]A$ to a $[]B$ through a function f of type **func**(A) B, spreading the work over *workers* goroutines. Because each result is written to its own index, no locking of *out* is needed.

⟨ The parallel-map function 3 ⟩ ≡

```
func pmap[A, B any](in []A, workers int, f func(A) B) []B {
    out := make([]B, len(in))
    var wg sync.WaitGroup
    jobs := make(chan int)
    ⟨ Start the worker goroutines 4 ⟩
    ⟨ Dispatch one job per input index 5 ⟩
    wg.Wait()
    return out
}
```

This code is used in section 2.

4. Each worker pulls indices off the *jobs* channel until it is closed, computing one result per index. The **defer** guarantees the wait group is released even if f panics.

⟨ Start the worker goroutines 4 ⟩ ≡

```
for w := 0; w < workers; w++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        for i := range jobs {
            out[i] = f(in[i])
        }
    }()
}
```

This code is used in section 3.

5. The dispatcher feeds every index and then closes the channel, which is the signal for the workers to finish.

⟨ Dispatch one job per input index 5 ⟩ ≡

```
for i := range in {
    jobs ← i
}
close(jobs)
```

This code is used in section 3.

6. Index.

⟨ Dispatch one job per input index 5 ⟩ Used in section 3
⟨ Start the worker goroutines 4 ⟩ Used in section 3
⟨ The parallel-map function 3 ⟩ Used in section 2

PMAP